

Auditoría — RaiseCode · Entrega Final

Fecha: 2026-05-17 · Rama auditada: **dev**

✓ Hecho

#	Item	Evidencia
1	README.md en raíz — descripción general, requisitos (Docker + Git), clonado, copia de .env , arranque con make up	README.md:1–24
2	Instrucciones SSL — generación y trust del cert para WSS, con variantes Linux / Mac / Windows / Docker	README.md:34–66
3	Tabla de comandos Make — logs, health, clean, ssl-certs, trust-cert, etc.	README.md:71–86
4	.env.example en la raíz — todas las variables que el código consume (DB_* , MONGO_* , REDIS_* , JWT_SECRET , WORKER_SECRET , VITE_API_URL ...) con placeholders y sin credenciales reales	.env.example:1–136
5	.env correctamente ignorado — reglas .env y .env.* en .gitignore:69–71 ; el fichero no está trackeado	.gitignore:69–71
6	docker-compose.yml de desarrollo — define nginx, frontend, api, worker, redis, mariadb, mongodb con healthchecks y hot-reload	docker-compose.yml:1–272
7	Redes Docker internas — tres redes separadas (frontend_network , backend_network , runner_network); los servicios internos no se exponen entre ellas	docker-compose.yml:248–255
8	Sin credenciales hardcodeadas en compose — todos los secrets usan \${VAR:-fallback_dev} , fallbacks marcados explícitamente como "not for production"	docker-compose.yml:90,94,144
9	Nginx como reverse proxy — gestiona HTTP→HTTPS, /api/ , /socket.io/ y frontend; rate-limit en /api/auth/* ; bloquea rutas sensibles (.env , .git , .sql , .bak ...)	nginx/conf.d/default.conf:1–217
10	Volúmenes persistentes en desarrollo — mariadb_data , mongodb_data , mongodb_config , redis_data declarados	docker-compose.yml:260–271
11	XP por dificultad implementado — execution.service.ts otorga 10/25/50 XP en la primera resolución e invoca evaluateForUser para logros	api/src/execution/execution.service.ts:139–152
12	Lógica de logros implementada — AchievementsService.evaluateForUser desbloquea 5 logros (Primer Paso, En Racha, Especialista, Explorador, Centurión)	api/src/achievements/achievements.service.ts:55–71

#	Item	Evidencia
13	Ranking funcional basado en XP — <code>RankingService</code> ordena por <code>xp DESC</code> ; frontend muestra columna "Experiencia" y "Completados" con selector por asignatura	<code>api/src/ranking/ranking.service.ts:9–32</code> , <code>frontend/src/ranking/components/RankingTable.tsx:33–34</code>
14	<code>execution_time</code> se recibe y se almacena — el backend guarda <code>time</code> : <code>BigInt(execution_time * 1000)</code> en <code>CompletedChallenges</code> y lo reenvía por WebSocket	<code>api/src/execution/execution.service.ts:119–163</code>
15	Timeout de ejecución centralizado — <code>TIMEOUT_MS = 30_000</code> como constante con nombre, no disperso	<code>frontend/src/challenges/solver/hooks/useExecution.ts:37</code>
16	<code>start:prod</code> correcto en backend — <code>node</code> <code>dist/main</code> (sin nodemon/ts-node) disponible en scripts de <code>package.json</code>	<code>api/package.json → scripts.start:prod</code>
17	Perfil de usuario mayoritariamente real — <code>completedCount</code> , <code>byLanguage</code> , <code>recentActivity</code> y <code>achievements</code> vienen de la API	<code>frontend/src/profile/ProfilePage.tsx:14–16</code>
18	Cabecera de reto sin mocks — <code>ChallengeDetailHeader</code> muestra datos reales del backend (título, dificultad, lenguaje, asignatura, creador)	<code>frontend/src/challenges/detail/components/ChallengeDetailHeader.tsx</code>

⚠ A medias / con problemas

1. README — sin documentación de seed ni migrate

El README no menciona `seed.sql` ni cómo ejecutar `prisma db seed`. Un corrector que clone y haga `make up` tendrá la BD vacía (sin logros, sin retos de ejemplo) a menos que conozca estos comandos:

```
# Paso previo obligatorio (las migraciones crean las tablas):
docker exec code_judge_api yarn prisma migrate deploy

# Opción 1 – SQL completo (recomendada):
docker exec -i code_judge_mariadb mysql -uroot -proot raisecode < seed.sql

# Opción 2 – Prisma seed (básico, sin Kotlin aún):
docker exec code_judge_api yarn prisma db seed
```

Qué falta: añadir sección "Base de datos / Seed inicial" en `README.md` con el orden correcto: migraciones primero, seed después.

2. `seed.sql` desactualizado — falta Kotlin

`seed.sql` fue actualizado por última vez el **2026-04-16**. Desde entonces se crearon 3 migraciones:

Migración	Qué añade
20260507000000	Columna <code>completedAt</code> en <code>CompletedChallenges</code>
20260507165825	Columna <code>createdAt</code> en <code>CompletedChallenges</code>
20260508000000	Inserta 'Kotlin' en la tabla <code>Language</code>

Impacto concreto: `seed.sql:21` solo inserta 4 lenguajes:

```
INSERT INTO Language (name) VALUES ('JavaScript'), ('Python'), ('Java'), ('TypeScript');
```

-- ❌ Falta Kotlin

El Prisma seeder (`seedCatalogues.ts`) sí incluye Kotlin. El worker tiene `KotlinRunner` registrado. El frontend lo maneja en `useChallengeSolver.tsx:15`. Si los profes usan `seed.sql`, Kotlin no existirá en la BD y cualquier reto en ese lenguaje fallará silenciosamente.

Qué falta: añadir `, ('Kotlin')` en `seed.sql:21`.

3. Seeder de Prisma — logros comentados

`api/prisma/seeds/seedCatalogues.ts:27–35` tiene el bloque `achievement.createMany` comentado con la nota "falla al crear logros". El `seed.sql` sí los incluye. La lógica de `AchievementsService` depende de que existan filas con nombres exactos (`Primer Paso, En Racha...`). Si alguien usa `prisma db seed` en lugar de `seed.sql`, los logros no se crearán y `evaluateForUser` nunca desbloqueará nada.

Qué falta: descomentar y reparar el bloque en `seedCatalogues.ts`.

4. Datos mock en perfil — racha y tiempo promedio

`ProfilePage.tsx:49–53` declara:

```
const mockStats = {
  currentStreak: "7 dias", // hardcoded
  avgTime: "125ms", // hardcoded
  bestStreak: "15 dias", // hardcoded
};
```

Estos tres valores se muestran en tarjetas de estadística visibles al usuario.

Qué falta: eliminar las tarjetas o conectarlas con datos reales de la API.

5. Mock visual de "Puntos" en creación de retos

`StepBasicInfo.tsx:149–174` renderiza un campo "Puntos" marcado con `// TODO: conectar con campo real cuando exista en el schema`. El schema de Prisma no tiene columna `points`; el valor no se persiste.

Qué falta: eliminar el campo visual o añadirlo al schema.

6. XP NO basado en tiempo de ejecución

La tarea interna dice "Sistema de experiencia basado en tiempo de ejecución". La implementación actual asigna XP por dificultad (Easy=10 / Medium=25 / Hard=50), ignorando `execution_time`. El campo `time` se guarda en `CompletedChallenges` pero no influye en el XP.

Qué falta: definir la fórmula de bonificación por velocidad y aplicarla en `api/src/execution/execution.service.ts:139–148`.

7. Tiempo de ejecución no se muestra al usuario

`ExecutionResult.execution_time` se recibe en el frontend (`useExecution.ts:58`) y llega a `SolverResultsPanel`, pero el componente no lo renderiza en ningún lugar. El modal de éxito tampoco lo muestra.

Qué falta: añadir en `SolverResultsPanel.tsx` dentro del bloque `status === 'done'`:

```
<Text color="gray.400" fontSize="xs">
  Tiempo total: {(result.execution_time * 1000).toFixed(1)} ms
</Text>
```

8. Bulk create sin selector de módulo/ciclo

`BulkUserForm.tsx` solo pide prefijo y cantidad. El backend (`users.service.ts:330`) asigna `defaultCourse.id` (primer curso en BD). El formulario individual de creación (`CreateUserModal.tsx`) sí tiene selector de curso.

Qué falta: añadir `<select>` de cursos en `BulkUserForm.tsx` y pasar `courseId` al DTO `CreateBulkGenericUsersDto`.

9. Servicios internos con puertos expuestos en el compose de desarrollo

El `docker-compose.yml` publica al host Redis (6379), MariaDB (3306) y MongoDB (27017). En desarrollo es habitual, pero si este compose se usa en producción (al no existir uno de prod), las BBDDs quedan accesibles desde fuera del servidor.

Qué falta: `docker-compose.prod.yml` (ver P1 más abajo).

✗ Pendiente

Ordenado por prioridad: primero lo que bloquea la entrega, después las features internas.

● BLOQUEAN LA ENTREGA

P1 — `docker-compose.prod.yml` — CRÍTICO

No existe. Es el requisito más explícito de infraestructura del enunciado.

Archivo a crear: `docker-compose.prod.yml` en la raíz con:

- `nginx` exponiendo **solo** puertos 80 y 443
 - `frontend` construido con `Dockerfile.prod` (build estático servido por Nginx)
 - `api` arrancando con `Dockerfile.prod` (multi-stage, `node dist/main`)
 - `worker`, `redis`, `mariadb`, `mongodb` **sin ports**: publicados al host
 - Todos los servicios en redes Docker internas
 - Secrets desde `.env`, sin fallbacks de desarrollo
-

P2 — `frontend/Dockerfile.prod`

No existe. Solo hay `Dockerfile.dev` que ejecuta el servidor Vite (no apto para producción).

Archivo a crear: `frontend/Dockerfile.prod`

```
# Stage 1 - build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2 - serve static
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
```

P3 — `api/Dockerfile.prod`

No existe. Sin él, el compose de producción no puede arrancar el backend compilado.

Archivo a crear: `api/Dockerfile.prod` con dos stages (build con `yarn build`, runtime con `node dist/main.js`).

P4 — Nginx config para producción

`nginx/conf.d/default.conf` hace proxy al servidor Vite dev (`upstream frontend { server frontend:5173; }`). En producción el frontend es estático y debe servirse desde disco.

Archivo a crear o adaptar: `nginx/conf.d/default.prod.conf` con:

```
location / {
    root /usr/share/nginx/html;
    try_files $uri $uri/ /index.html;
}
```

P5 — Documentación de seed y migrate en README

Sin esto, el corrector no puede poblar la BD ni sabe en qué orden ejecutar los comandos.

Archivo a modificar: `README.md` — añadir sección:

```
## Base de datos – Seed inicial

# 1. Aplicar migraciones (crea las tablas):
docker exec code_judge_api yarn prisma migrate deploy

# 2. Poblar con datos de ejemplo:
docker exec -i code_judge_mariadb mysql -uroot -proot raisecode < seed.sql

# NOTA: En desarrollo, el comando anterior puede sustituirse por:
# docker exec code_judge_api yarn prisma db seed
# (No incluye Kotlin hasta que se corrija el seeder)
```

🟡 FEATURES DEL EQUIPO PENDIENTES

P6 — Añadir Kotlin a `seed.sql`

Archivo a modificar: `seed.sql:21`

```
-- Cambiar:
INSERT INTO Language (name) VALUES ('JavaScript'), ('Python'), ('Java'), ('TypeScript');
-- Por:
INSERT INTO Language (name) VALUES ('JavaScript'), ('Python'), ('Java'), ('TypeScript'), ('Kotlin');
```

P7 — Descomentar seeder de logros en Prisma

Archivo a modificar: `api/prisma/seeds/seedCatalogues.ts:27–35` — descomentar y reparar el bloque `achievement.createMany`.

P8 — Botón de desactivar registro

No existe en ningún sitio (ni backend ni frontend). La ruta `POST /api/auth/register` está siempre abierta.

Archivos a tocar:

- Backend: nuevo endpoint `PATCH /api/admin/settings` con flag `registrationEnabled`
- Frontend: toggle en `AdminDashboardPage.tsx`

P9 — Botón de parar ejecución

`ChallengeEditorToolbar.tsx` solo tiene el botón "Ejecutar" (deshabilitado mientras corre). No hay forma de cancelar.

Archivos a tocar:

- `frontend/src/challenges/solver/hooks/useExecution.ts` — añadir función `abort()`
 - `frontend/src/challenges/solver/components/ChallengeEditorToolbar.tsx` — añadir botón "Cancelar" visible cuando `isRunning === true`
-

P10 — XP basado en tiempo de ejecución

Archivo a tocar: `api/src/execution/execution.service.ts:139-148` — definir fórmula de bonus por velocidad y sumarla a `xpToAdd`.

P11 — Mostrar tiempo de ejecución en UI

Archivo a tocar: `frontend/src/challenges/solver/components/SolverResultsPanel.tsx` — renderizar `result.execution_time` dentro del bloque `status === 'done'`.

P12 — Selector de módulo/ciclo en bulk create

Archivos a tocar:

- `api/src/users/dto/create-bulk-generic-users.dto.ts` — añadir campo `courseId`
 - `frontend/src/admin/pages/users/components/BulkUserForm.tsx` — añadir `<select>` de ciclos formativos
-

Resumen de prioridades

P1	<code>docker-compose.prod.yml</code>	← BLOQUEANTE (entrega)
P2	<code>frontend/Dockerfile.prod</code>	← BLOQUEANTE (entrega)
P3	<code>api/Dockerfile.prod</code>	← BLOQUEANTE (entrega)
P4	nginx config para producción	← BLOQUEANTE (entrega)
P5	Documentar seed/migrate README	← Muy recomendable
P6	Añadir Kotlin a <code>seed.sql</code>	← Fácil, 1 línea
P7	Descomentar seeder de logros	← Fácil
P8	Desactivar registro (admin)	← Feature interna
P9	Botón parar ejecución	← Feature interna
P10	XP por tiempo de ejecución	← Feature interna
P11	Mostrar tiempo en UI	← Fácil, ~5 líneas
P12	Selector módulo en bulk create	← Feature interna